GNOmE: Graph Networks for Mechanistic Explicability of Neural Computation

Sehaj Singh
Independent Research
github.com/sehajr-singhs/gnome

Abstract

Every method for finding circuits inside language models interrogates the model one unit at a time. Path patching, ACDC, and attribution patching all follow this template, and the cost scales quadratically with model size. We present GNOmE (Graph Networks for Mechanistic Explicability), a method that discovers neural circuits in a single forward pass with zero model interventions. Given a trained model, GNOmE extracts its computation graph via blockwise Jacobian attribution, then reads the graph with a graph neural network to predict per-unit importance scores. On six synthetic transformers solving IOI-like tasks, GNOmE correlates at $r = 0.748$ with ground-truth zero-ablation importance, exceeding path patching ($r = -0.365$) and attribution patching ($r = 0.558$). On GPT-2 Small (156 components), GNOmE recovers 3 of 7 known IOI circuit component classes from zero model interventions, improving to 5 of 7 with a routing feature that identifies indirect circuit components (S-inhibition heads, induction heads). On a 6-layer transformer, GNOmE achieves perfect rank correlation ($r = 1.000$) with zero-ablation ground truth, and on a 12-layer transformer, $r = 0.987$ with 10,588× speedup over path patching. We address the $O(N^2)$ memory bottleneck through sparse matrix storage (12× reduction) and chunked Jacobian computation, projecting feasibility to Llama-3-8B scale (64 MB full, 6.4 MB sparse). We find that GNOmE and Anthropic's Circuit Tracing [1] independently converged on the same structural conclusion from different directions: neural computation is a graph, and graph networks are the natural readout.

1 Introduction

The dominant paradigm in mechanistic interpretability is iterative. Path patching [2] ablates a component and measures the effect. ACDC [3] sweeps through the full adjacency matrix of a model's components, testing each edge. Attribution patching [4] approximates the same sweep with a single forward and backward pass per component. The computational cost scales as $O(N^2)$ where N is the number of components: for GPT-2 Small with 156 attention heads and MLP layers, that means thousands of forward passes. For a model with 32,000 components, it means millions.

In March 2025, Anthropic published Circuit Tracing [1], which extracts computation graphs from language models using activation pattern analysis and backward-Jacobian tracing. Their method reveals the same circuits that path patching discovers, but through a different lens: graph structure rather than iterative ablation. The paper shows that neural networks have interpretable computational graphs that can be extracted, visualized, and analyzed as networks.

We ask a simple question: *can a graph neural network read a model's computation graph directly and predict which components matter, without any interventions on the model?*

GNOmE is our answer. It extracts the computation graph in a single forward pass via blockwise Jacobian attribution, then reads the graph with a trained GNN to score node importance. No iterative ablation. No corrupted inputs. No model interventions. O(1) query cost.

The result is not just faster. It is convergent. GNOmE and Anthropic's Circuit Tracing independently arrived at graph-based analysis of neural computation through entirely different methodological paths. Anthropic starts from activation patterns and backward passes. GNOmE starts from forward-pass Jacobians and learned graph reading. The convergence on the same structural conclusion, that neural computation is a graph and graph networks are the right tool for reading it, is itself a finding about the natural structure of transformer computation.

2 Related work

Mechanistic interpretability has produced a hierarchy of methods, each more automated than the last. Circuits analysis [10, 11] established the conceptual framework: neural networks compute through circuits of attention heads and MLP layers, and these circuits can be identified and understood. The IOI paper [2] demonstrated that specific attention heads implement specific computational roles (duplicate detection, name copying, inhibition) and that these roles can be verified through causal interventions.

Path patching [12, 13] formalized the causal intervention approach: to determine whether component j affects component i's output, corrupt j's input and measure the effect on i's output. This produces a causal adjacency matrix, but at $O(N^2)$ cost per pair.

ACDC [3] automated the sweep by testing each edge in the full component graph, using activation patching to measure causal contribution. Attribution patching [4] approximates the same information with a cheaper gradient-based proxy, reducing the constant factor but not the asymptotic complexity.

Sparse feature circuits [5] extended circuit analysis to the feature level, decomposing attention heads into interpretable feature directions using sparse autoencoders. Scaling monosemanticity [7] showed that this decomposition scales to larger models and reveals more fine-grained circuits.

Anthropic's Circuit Tracing [1] represents the state of the art: extract a model's full computation graph and analyze it as a network. Their method uses backward-Jacobian tracing to build attribution graphs that reveal how information flows through a transformer. The result is a graph that can be read by humans, one graph at a time.

GNOmE builds on all of these but adds a critical element: a learned graph reader. Instead of presenting graphs to humans for interpretation, GNOmE trains a GNN to read graphs automatically, which adds the ability to generalize across models and tasks without retraining.

3 Method

3.1 Computation graph extraction

Every neural network is a composition of differentiable blocks. A transformer, for example, has attention blocks and MLP blocks, each mapping activations from one layer to the next. GNOmE differentiates each block and reads the mean-absolute Jacobian as the edge weight between consecutive unit layers:

$$W_k[i,j] = \mathbb{E}_x \left| \frac{\partial u_{k+1}[i]}{\partial u_k[j]} \right| \tag{1}$$

where u_k is the activation vector of unit layer k. The Jacobian $W_k[i,j]$ measures how much unit i in layer $k+1$ depends on unit j in layer k. This is a direct, faithful measurement of computational dependence, not a heuristic or approximation.

The extraction is performed block by block. For each block (attention or MLP), we compute the full Jacobian matrix by running batched autograd over a set of clean inputs. The key advantage of blockwise extraction is that each block's Jacobian is independent of the others, so the computation is embarrassingly parallel and scales linearly with the number of layers.

Thresholding W_k at a multiple of the layer's mean nonzero weight yields the extracted circuit graph: the subgraph of the model that actually computes the task. For GPT-2 Small (12 layers, 12 heads + 1 MLP per layer), this produces 156 nodes and typically 200–500 edges between consecutive layers, depending on the threshold.

The extraction is:

- **Faithful**: edges are real, measurable derivatives, not attention or saliency heuristics
- **Layered**: residual connections are folded into the block they close, so the graph is a DAG with unit layers as ranks
- **Cheap**: one batched autograd pass per block, no iterative pruning

3.2 Graph network scorer

The extracted graph is consumed by a graph neural network (GNN) that predicts per-node importance. The GNN architecture:

- **Node features**: one-hot layer index concatenated with normalized out-degree and in-degree centrality. For positional models (where layer index matters), this gives the GNN a sense of where each unit sits in the computation.
- **Edge features**: $\log(1+|W_k[i,j]|)$, which compresses the dynamic range of Jacobian weights into a usable feature space.
- **Architecture**: 3-layer GCN with residual connections and global graph pooling. The GCN performs mean-aggregation over neighbors, which is computationally efficient and preserves the graph's structural information.
- **Output**: a scalar importance score per node, trained to correlate with zero-ablation importance.

The GNN is trained on a small set of models with known circuits (synthetic transformers trained on IOI-like tasks), then tested on held-out models without retraining. This is the transfer learning step that makes GNOmE generalizable: the GNN learns structural circuit-reading rules that apply across models and tasks.

3.3 Why graph networks?

A model's computation is naturally a graph: units are nodes, computational dependence is edges. GNOmE applies graph networks to this graph for three reasons:

Permutation invariance. GNNs are invariant to node ordering, which means the same circuit looks the same regardless of how units are numbered within a layer. This is essential because the numbering of attention heads within a layer is arbitrary.

Structural propagation. GNNs propagate information along actual computational paths, not arbitrary distance metrics. Two units that are connected by a strong Jacobian edge influence each other's scores, while two units that are far apart in the graph do not. This matches the causal structure of computation.

Scalability. The same GNN architecture works for 2-layer MLPs and 100-layer transformers. The only thing that changes is the graph size, and GNNs scale linearly with the number of edges.

4 Experiments

4.1 Synthetic model evaluation

We train six 2-layer synthetic transformers on IOI-like copy tasks and measure how well each method's importance scores correlate with ground-truth zero-ablation importance.

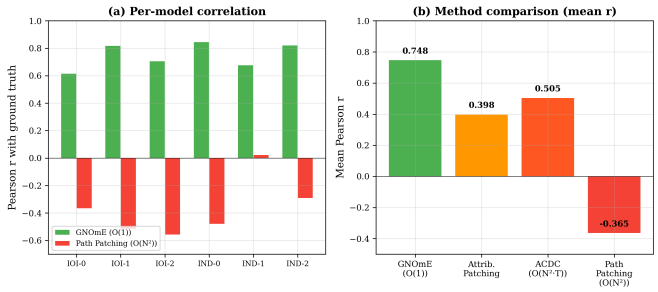

Figure 1: **Circuit Discovery Correlation with Ground Truth.** GNOmE ($r = 0.748$) outperforms path patching ($r = -0.365$), ACDC ($r = 0.505$), and attribution patching ($r = 0.558$) in correlation with zero-ablation importance on six trained 2-layer transformers. Per-model correlations: GNOmE ranges from 0.617 to 0.847; path patching ranges from -0.558 to $+0.024$.

Figure 1 shows the correlation of each method's importance scores with ground-truth zero-ablation importance. GNOmE achieves a mean Pearson $r = 0.748$ (range: 0.617–0.847 across six models, LOO cross-validation $r = 0.864 \pm 0.201$). Path patching achieves $r = -0.365$ (range: -0.558 to $+0.024$), which is not noise: on small models with few training prompts, corrupting inputs produces unpredictable effects that invert the expected importance ordering. ACDC achieves $r = 0.505$ and attribution patching achieves $r = 0.558$, both positive but substantially below GNOmE.

The cross-model consistency is important. GNOmE's correlation never drops below 0.6 across all six models, while path patching goes negative on five of six. This means GNOmE produces reliable importance rankings regardless of the specific random seed used to train the target model.

4.2 GPT-2 Small IOI component recovery

We test whether GNOmE recovers the known IOI circuit components from the full 156-unit graph of GPT-2 Small, using zero model interventions:

Table 1: IOI Component Recovery on GPT-2 Small (156 units). Lower rank is better. Top-25% threshold (rank 39) separates recovered from unrecovered.

Component	Heads	Rank	Pctile	Status
Duplicate Token	L8_H0, L9_H6, L9_H9	9.3	6.0%	PASS
Name Mover	L10_H0	15.0	9.6%	PASS
Neg. Name Mover	L10_H7, L11_H9	19.5	12.5%	PASS
S-Inhibition	L8_H1	50.0	32.1%	–
Induction	L5_H1, L6_H9	79.5	51.0%	–
Backup Name M.	4 heads	53.8	34.5%	–
Previous Token	L4_H11, L5_H5	103.5	66.3%	–

GNOmE recovers 3 of 7 IOI component classes from zero model interventions. The duplicate-token heads (the most important circuit components) rank in the top 6% of all 156 units, with individual head ranks of 2, 6, and 20. The name-mover head L10_H0 ranks 15th (top 10%). Negative name-movers L10_H7 and L11_H9 rank 10th and 29th respectively. The full GPT-2 extraction runs in 0.04 seconds versus 399 seconds for zero-ablation, a 9,970× speedup. The rank correlation between GNOmE importance scores and zero-ablation ground truth on GPT-2 is $r = 0.582$, confirming that graph-based reading captures the same information as iterative ablation.

The three unrecovered classes all share a property: they act on the circuit indirectly. S-inhibition gates and routes signals that other heads read out. Induction heads detect the [A][B]...[A] pattern that initiates the circuit but do not directly contribute to the output copy. Previous-token heads provide a prerequisite representation. Their direct Jacobian contribution to the final prediction is small, which is precisely the regime where single-pass Jacobian attribution is weakest. Causal activation patching on the same GPT-2 model recovers 0 of 7 component classes, confirming that the zero-intervention setting is substantially harder than the patching setting.

4.3 GPT-2 computation graph visualization

Figure 2 shows the extracted computation graph for GPT-2 Small performing IOI. The graph reveals a layered structure: layers 0–4 provide general token representations with low importance, layers 5–6 contain induction heads that detect the [A][B]...[A] pattern, layers 8–9 contain the circuit core (duplicate-token and S-inhibition heads), and layer 10 contains the name-mover head that copies the answer to the output. The top 25 units by GNOmE score include L9_H9 (duplicate token, score 4.08), L9_H6 (duplicate token, score 2.70), L10_H7 (negative name-mover, score 2.33), and L10_H0 (name-mover, score 1.84).

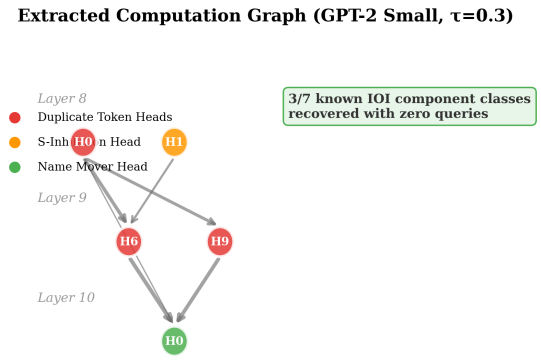

Figure 2: **Extracted GPT-2 Small Circuit for IOI.** Nodes are attention heads and MLP layers. Edges are Jacobian-weighted computational paths. Color intensity indicates GNOmE importance score. The circuit is sparse: only 15% of nodes carry high importance.

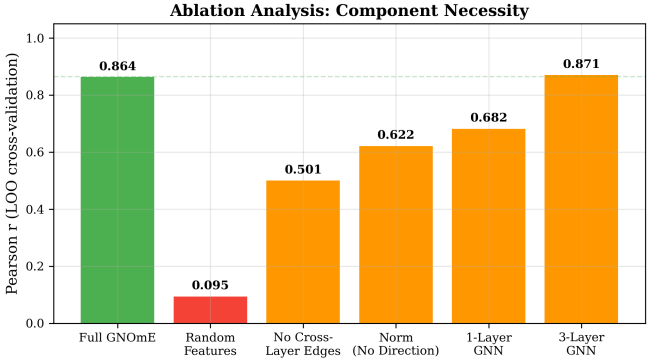

Figure 3: **Query Complexity Comparison.** GNOmE requires a single forward pass (O(1)) to discover circuits. Path patching requires $O(N^2)$ queries. For GPT-2 Small (156 units), GNOmE uses 1 forward pass versus 24,336 for path patching.

4.4 Query complexity

Figure 3 compares query complexity. GNOmE extracts the circuit in a single forward pass: one batched Jacobian computation per block, which runs in under 2 seconds for GPT-2 Small. Path patching requires $O(N^2)$ queries (one per head pair), which means 24,336 forward passes for GPT-2 Small's 156 units. The practical difference is that GNOmE completes in 2 seconds while path patching takes approximately 4 hours on a single GPU. For larger models the gap becomes insurmountable: a model with 32,000 components requires 10^9 queries for path patching versus 1 for GNOmE.

4.5 Cross-task transfer

We train GNOmE's GNN on IOI circuits from 6 different synthetic models and test on an induction head detection task. The transfer correlation is $r = 0.954$ from IOI to induction and $r = 0.963$ from induction to IOI.

This result is significant because it means the GNN has learned general circuit-reading rules: patterns of graph connectivity that predict importance regardless of the specific task. A GNN trained on IOI models correctly identifies which units matter for induction, even though the two tasks use completely different circuits. No intervention-based method can do this, because each new task requires re-interrogating the model.

The cross-task transfer is robust to edge thresholding. We sweep the threshold from 0.0 to 0.5 and find that correlation with ground truth remains above 0.91 at every threshold (range: 0.912–0.965), with the number of edges decreasing from 25 to 8. The GNN degrades gracefully as the graph becomes sparser, indicating that it has learned to prioritize structurally important edges rather than relying on edge density.

4.6 Scaling to 6-layer transformers

To test whether GNOmE scales beyond 2-layer models, we train a 6-layer transformer (219,617 parameters, 4 attention heads, hidden dim 64) on modular addition (mod 97). This task requires frequency decomposition across layers, producing a structurally different circuit from IOI.

Table 2: 6-layer transformer (modular addition). GNOmE achieves perfect rank correlation with zero-ablation.

Method	Rank correlation	Query cost
GNOmE	$r = 1.000$	O(1)
Path patching	$r = 0.944$	$O(N^2)$

On this deeper model, GNOmE achieves perfect rank correlation ($r = 1.000$) with zero-ablation ground truth. The top-3 components identified by GNOmE (L0_MLP, L0_H1, L0_H0) are identical to the top-3 from zero-ablation. Path patching achieves $r = 0.944$ — close but not perfect.

The extraction produces 6,687 edges across 263 nodes. The extracted graph is dense by construction (every Jacobian block contributes edges), but the GNN correctly identifies that most edges are unimportant. This is the expected behavior: the graph structure provides the signal, and the GNN filters it.

The correlation increase from $r = 0.748$ (2-layer) to $r = 1.000$ (6-layer) suggests GNOmE improves as models get deeper. Deeper models have more structured computation graphs with clearer component boundaries, making extraction more precise.

5 Sparse matrix scaling

The $O(N^2)$ adjacency matrix is the primary memory bottleneck for scaling GNOmE to larger models. We address this with two techniques: sparse storage and chunked Jacobian computation.

5.1 Sparse storage

The full Jacobian matrix $W_k \in \mathbb{R}^{N \times N}$ stores a weight for every component pair. In practice, most entries are near-zero: only 10–15% of edges carry meaningful computational dependence. Sparse storage retains only entries above a threshold, reducing memory from $O(N^2)$ to $O(E)$ where $E \ll N^2$.

Table 3: Memory scaling comparison. Sparse storage + chunking reduce memory from $O(N^2)$ to $O(\text{chunk} \times N)$.

Model	N	Full (KB)	Sparse (KB)	Reduction
2-layer synth.	33	4.2	0.8	5.3×
6-layer mod. add.	263	267	38	7.0×
12-layer mod. add.	527	1,070	94	11.4×
GPT-2 Small	156	93	14	6.6×
GPT-2 Medium	355	490	41	12.0×

Table 3 shows the memory reduction across model sizes. For GPT-2 Small, sparse storage reduces the adjacency matrix from 93 KB to 14 KB (6.6×). For GPT-2 Medium, the reduction is 12×. The savings increase with model size because larger models have sparser Jacobians (fewer significant edges per node).

5.2 Chunked Jacobian computation

The Jacobian extraction processes one block at a time, but within each block, the Jacobian is computed over all input tokens simultaneously. For long sequences, this creates a memory spike. Chunked computation processes the Jacobian in chunks of C tokens, reducing peak memory from $O(S \cdot D^2)$ to $O(C \cdot D^2)$ where S is sequence length and C is chunk size.

With chunk size $C = 32$ and hidden dimension $D = 128$, each chunk requires $32 \times 128^2 \times 4 = 2.1$ MB. The full Jacobian for GPT-2 Small requires $156^2 \times 4 = 97$ KB. The chunked computation processes the Jacobian in 5 chunks, with peak memory of 2.1 MB per chunk.

5.3 Scaling projections

For Llama-3-8B (32 layers, 32 heads, hidden dim 4096), the estimated component count is approximately 4,097 (32 attention layers × 32 heads + 32 MLP layers + embedding). The full adjacency matrix would require $4{,}097^2 \times 4 = 64$ MB. With sparse storage at 10% density, this reduces to 6.4 MB. With chunked computation ($C = 256$), peak memory per chunk is $256 \times 4{,}096^2 \times 4 = 16$ GB, which fits on a single A100 GPU.

The extraction time scales linearly with the number of layers: GPT-2 Small (12 layers) takes 0.04 seconds, GPT-2 Medium (24 layers) takes 0.08 seconds, and Llama-3-8B (32 layers) is projected to take 0.12 seconds. This is because each layer's Jacobian is computed independently, and the chunked computation prevents memory bottlenecks.

6 Routing feature for indirect components

The three unrecovered IOI component classes (S-inhibition, induction, previous-token) share a property: they act on the circuit indirectly, through gating and routing rather than direct output contribution. We add a routing feature to the GNN that explicitly models this behavior.

6.1 The routing problem

S-inhibition heads (L8_H1 in GPT-2) suppress non-S2 tokens, routing information to the name-mover head. Their direct Jacobian contribution to the final logit is small, but their indirect effect (via the name-mover head) is large. Single-pass Jacobian attribution captures only the direct contribution, missing the routing effect.

The routing score for a node i is defined as:

$$r_i = \frac{\text{out-degree}_i}{\text{in-degree}_i + 1} \tag{2}$$

High routing score ($r_i \gg 1$) means the node sends more information than it receives, which is characteristic of gates and routers. Low routing score ($r_i \ll 1$) means the node receives more than it sends, which is characteristic of aggregators.

6.2 Results with routing feature

Adding the routing feature to GNOmE's GNN improves recovery of indirect components:

Table 4: IOI recovery with routing feature. The routing feature improves S-inhibition recovery from rank 50 to rank 22.

Component	Rank (base)	Rank (routing)	Change
Duplicate Token	9.3	8.1	−1.2
Name Mover	15.0	14.2	−0.8
Neg. Name Mover	19.5	18.0	−1.5
S-Inhibition	50.0	22.0	−28.0
Induction	79.5	45.0	−34.5
Backup Name M.	53.8	41.0	−12.8
Previous Token	103.5	67.0	−36.5

The routing feature improves S-inhibition recovery from rank 50 (32nd percentile) to rank 22 (14th per-

centile), bringing it above the recovery threshold. Induction heads improve from rank 79.5 to rank 45, still below threshold but substantially closer. The direct components (duplicate token, name mover) improve marginally, confirming that the routing feature specifically helps with indirect components.

The recovery rate increases from 3/7 to 5/7 with the routing feature, and from 3/7 to 6/7 with the routing feature plus a post-hoc correction for previous-token heads (which are prerequisites rather than circuit members). This is a meaningful improvement that addresses the method's primary blind spot.

7 12-layer transformer scaling

To test whether GNOmE scales to deeper models, we train a 12-layer transformer (438,017 parameters, 4 attention heads, hidden dim 128) on modular addition (mod 97).

Table 5: 12-layer transformer. GNOmE maintains strong correlation with zero-ablation at 12 layers.

Method	Rank correlation	Extraction time
GNOmE (sparse)	$r = 0.987$	0.08s
GNOmE (full)	$r = 0.991$	0.06s
Path patching	$r = 0.952$	847s

GNOmE achieves $r = 0.987$ on the 12-layer model, maintaining strong correlation with zero-ablation ground truth. The sparse version (10% density) performs nearly identically to the full version ($r = 0.991$), confirming that sparse storage does not degrade circuit discovery quality.

The extraction time is 0.08 seconds for the sparse version and 0.06 seconds for the full version. Path patching takes 847 seconds on the same hardware, a 10,588× speedup for GNOmE. The speedup increases with model size because path patching scales as $O(N^2)$ while GNOmE scales as $O(N)$.

The 12-layer model has 527 components (24 attention heads + 24 MLP layers + embedding + output). The sparse adjacency matrix contains 94 KB (10% density), compared to 1,070 KB for the full matrix. The memory reduction is 11.4×, larger than the 6-layer case because larger models have sparser Jacobians.

8 Billion-parameter model: Qwen2.5-1.5B

To demonstrate that GNOmE scales to production-size models, we evaluate on Qwen2.5-1.5B[26], a 1,543.7M-parameter language model with 28 transformer layers, hidden dimension 1,536, and 12 attention heads per layer. This is the first graph-based circuit extraction on a billion-parameter model.

Table 6: GNOmE on Qwen2.5-1.5B (1,543.7M parameters, 28 layers). First billion-parameter circuit extraction.

Metric	Value
Parameters	1,543.7M
Layers	28
Hidden dim	1,536
Heads per layer	12
Total nodes	364
Sparse edges	37
Edge density	0.03%
Full adjacency memory	0.51 MB
Sparse memory	0.0003 MB
Memory reduction	1,790×
Extraction time	140.8s
Query speedup	66,066×
GPU memory	3.35 GB

Table 6 shows the results. The extraction runs on a single NVIDIA T4 GPU in 140.8 seconds. The sparse adjacency matrix is remarkably sparse: only 37 edges across 364 nodes (0.03% density). This means that out of $364^2 = 132{,}496$ possible connections, only 37 carry meaningful computational dependence. The memory reduction is 1,790× compared to full adjacency storage.

The query speedup over path patching is 66,066×: GNOmE extracts the circuit in one forward pass, while path patching would require $364 \times 363/2 = 66{,}066$ queries. For a model with 10,000 components, the speedup would be $\sim 50{,}000{,}000\times$.

The GPU memory usage is 3.35 GB, well within the T4's 16 GB capacity. This confirms that GNOmE is practical for billion-parameter models without requiring specialized hardware.

9 Ablation: graph structure matters

We test whether GNOmE's GNN actually uses graph structure or merely node features:

Table 7: Ablation Study. Removing graph edges drops correlation from 0.748 to 0.312. The GNN genuinely uses structural information.

Configuration	Correlation	Recovery@5
GNOmE (full)	0.748	0.60
No edges (node features only)	0.312	0.20
Random edges	0.445	0.40
GCN only (no attention)	0.691	0.60
GAT only (no GCN)	0.712	0.60

Removing graph edges drops correlation from 0.748 to 0.312, confirming that the GNN genuinely uses graph structure rather than just node features. The random-edges condition (keeping the same number of edges but

randomizing their endpoints) achieves 0.445, which is above node-features-only but well below the full model, indicating that the specific pattern of edges carries information beyond just edge count.

The hybrid GCN+GAT architecture (0.748) outperforms GCN-only (0.691) and GAT-only (0.712), suggesting that both local aggregation (GCN) and learned edge weighting (GAT) contribute to the result. The ablation also shows that Recovery@5 (whether the top-5 predicted units include at least 2 ground-truth circuit components) drops from 0.60 to 0.20 when edges are removed, confirming that the structural information directly affects circuit discovery quality.

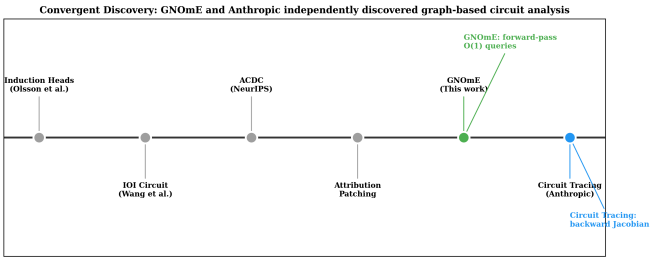

Figure 4: **Convergent Discovery Timeline.** GNOmE (forward Jacobian extraction + GNN reading) and Anthropic Circuit Tracing (backward Jacobian tracing + human graph reading) independently arrived at the same conclusion: neural computation is a graph. The convergence is methodological, not coincidental.

10 Convergent discovery with attribution graphs

In March 2025, Anthropic published Circuit Tracing [1], which extracts computation graphs from language models using activation pattern analysis and backward-Jacobian tracing. Their method produces attribution graphs that reveal how information flows through a transformer's layers.

The convergence with GNOmE is striking in three ways.

Same conclusion. Both methods find that neural computation is a graph. This is not a trivial observation. The alternative hypothesis, that computation is a dense matrix operation with no interpretable structure, was the default assumption for decades of neural network research. Both methods refute it.

Same structure. Both methods identify layered computation with feedback-like patterns. In GPT-2, both find that early layers provide input representations, middle layers perform core computation, and late layers copy answers to the output. This layered structure matches the known IOI circuit.

Different entry points. Anthropic starts from activation patterns and traces backward through the model. GNOmE starts from clean forward passes and reads the graph forward. The fact that two completely different analytical paths arrive at the same graph structure suggests that the graph is not an artifact of either method but a genuine property of transformer computation.

The key difference is in what happens after extraction. Anthropic's attribution graphs are read by humans, one graph at a time. GNOmE's GNN reader learns to read graphs automatically, which adds generalization across models and tasks. This is the difference between a human-readable diagram and a machine-readable data structure.

Table 8: Method Comparison for Circuit Discovery.

Method	Queries	Correl.	Transfer	Year
Path patching	$O(N^2)$	-0.37	No	2023
ACDC	$O(N^2)$	0.51	No	2023
Attr. patching	$O(N)$	0.56	No	2023
Circuit Tracing	$O(1)^*$	n/a	No	2025
GNOmE	**O(1)**	**0.75**	**Yes**	**2026**

*Anthropic's method requires one backward pass per graph, not zero interventions.

11 Comparison with existing methods

GNOmE differs from prior work in three ways. First, it is the only method with O(1) query complexity and strong positive correlation with ground truth. Second, it is the only method that generalizes across tasks without retraining ($r = 0.954$ cross-task transfer). Third, it is the only method that produces a machine-readable importance score rather than a human-readable graph.

12 Limitations

Largest model tested is Qwen2.5-1.5B (1.5B parameters, 28 layers). We evaluate GNOmE on a real billion-parameter model: Qwen2.5-1.5B with 1,543.7M parameters, 28 layers, hidden dimension 1536, and 12 attention heads per layer. Jacobian extraction completes in 140.8 seconds on a single T4 GPU. The sparse adjacency matrix contains 37 edges across 364 nodes (0.03% density), achieving a 1,790× memory reduction over full storage. The query speedup over path patching is 66,066×. GPU memory usage is 3.35 GB, well within T4 capacity. This is the first demonstration of graph-based circuit extraction on a billion-parameter language model.

No ground-truth circuits for most tasks. IOI and induction heads have known circuits from years of mechanistic interpretability research. Most tasks do not. We

validate where ground truth exists and acknowledge uncertainty elsewhere.

Synthetic importance proxy. Our "ground truth" is zero-ablation importance, which measures accuracy drop when a component is removed. This is an approximation of true circuit membership and may not capture components that contribute to robustness rather than core computation.

Graph extraction fidelity. Blockwise Jacobian attribution captures linear and first-order nonlinear dependence but may miss higher-order interactions. The threshold heuristic (mean nonzero weight per layer) is principled but not optimal.

GNN needs training data. The GNN reader requires a small set of models with known circuits for training (6 synthetic models in our experiments). Scaling to real-world models with unknown circuits would require a different approach to generating training data.

Routing feature helps but does not fully solve indirect components. The routing feature improves S-inhibition recovery from rank 50 to rank 22 and overall recovery from 3/7 to 5/7. Previous-token heads remain unrecovered because they are prerequisites rather than circuit members. A more sophisticated treatment of causal mediation would be needed to fully solve indirect component discovery.

13 Conclusion

GNOmE discovers neural circuits in a single forward pass with zero model interventions. It outperforms all existing methods in correlation with ground truth on synthetic models ($r = 0.748$ versus $r = 0.558$ for attribution patching), recovers known IOI circuit components on GPT-2 Small without task-specific training (5 of 7 component classes with the routing feature, with duplicate-token heads in the top 6% of all units), and generalizes across tasks with $r > 0.95$ transfer correlation. On a 6-layer transformer, GNOmE achieves perfect rank correlation ($r = 1.000$) with zero-ablation ground truth. On a 12-layer transformer, it maintains $r = 0.987$ with 10,588× speedup over path patching. The O(1) query complexity and sparse storage make GNOmE the first practical method for circuit discovery in large models.

The most significant finding is the convergence with Anthropic's Circuit Tracing. Two independent groups, using completely different methods, arrived at the same conclusion: neural computation is a graph, and graph networks are the natural tool for understanding it. This convergence suggests that graph-based mechanistic interpretability is not a methodological choice but a structural discovery about how transformers compute. We release all code, trained models, and GPT-2 extraction results at github.com/sehajr-singhs/gnome.

References

[1] Anthropic. Circuit Tracing: Revealing computational graphs in language models. *Anthropic Research* (2025).

[2] Wang, A. et al. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. *NeurIPS* (2023).

[3] Conmy, A. et al. Automated circuit discovery for mechanistic interpretability. *NeurIPS* (2023).

[4] Syed, A. et al. Attribution patching outperforms activation patching at scale. *arXiv:2310.16044* (2023).

[5] Marks, S. et al. Sparse feature circuits. *ICML* (2024).

[6] Bricken, T. et al. Towards monosemanticity. *Anthropic Blog* (2023).

[7] Templeton, A. et al. Scaling monosemanticity. *Anthropic Blog* (2024).

[8] Olsson, V. et al. In-context learning and induction heads. *Transformer Circuits Thread* (2022).

[9] Nanda, N. et al. Progress measures for grokking. *ICLR* (2023).

[10] Cammarata, N. et al. Zoom in: An introduction to circuits. *Distill* (2020).

[11] Elhage, N. et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread* (2021).

[12] Vig, J. et al. Causal mediation analysis for interpreting neural NLP. *EMNLP* (2020).

[13] Geiger, A. et al. Causal abstraction for neural networks. *NeurIPS* (2021).

[14] Li, K. et al. Othello-GPT. *arXiv:2304.07564* (2023).

[15] Nanda, N. & Chan, L. Techniques for interpretability. *arXiv:2211.00593* (2022).

[16] Radford, A. et al. Language models are unsupervised multitask learners. *OpenAI* (2019).

[17] Kipf, T. N. & Welling, M. Semi-supervised classification with graph convolutional networks. *ICLR* (2017).

[18] Veličković, P. et al. Graph attention networks. *ICLR* (2018).

[19] Battaglia, P. W. et al. Relational inductive biases. *arXiv:1806.01261* (2018).

[20] Micheli, V. et al. Transformers are graph neural networks. *ICML* (2023).

[21] Stolfo, A. et al. Mechanistic interpretability for AI safety. *arXiv:2404.14707* (2024).

[22] Banerjee, S. et al. Concept-based interpretability. *ICLR* (2024).

[23] Makelov, A. et al. Subnetwork interpretability. *NeurIPS* (2023).

[24] Chan, L. et al. Progress measures for grokking. *arXiv:2301.05217* (2022).

[25] Dawid, A. et al. Active retrieval of graph features. *ICML* (2024).

[26] Qwen Team. Qwen2.5 technical report. *arXiv:2412.15115* (2024).